

TEST PLAN - API Resources (Klavi/Open Finance Brazil)

Version: 1.0

Date: 06/24/2026

Author: Rodrigo Souza

Summary

- 1 - Goals
 - 2 - Version
 - 3 - Scope
 - 4 - Test Environments
 - 5 - Project Management & Agile Approach
 - 6 - Risks
 - 7 - Test Strategy
 - 8 - CI/CD Pipeline
 - 9 - Tools and Infrastructure
 - 10 - Activities & Estimates
 - 11 - Test Matrix
 - 12 - Communication Plan & Reports
-

1. Goals

This test plan aims to ensure that the API Resources v3.0.0 works correctly and securely according to the Open Finance Brazil specification. We want to validate that:

The API returns accurate data and follows the specified schema, so customers can trust the resource information they receive.

All data fields like resourceId, type, and status are properly validated and formatted according to the requirements.

Security is solid - OAuth 2.0 authentication works correctly and the RESOURCES_READ permission is properly enforced, so only authorized institutions can access this data.

The API handles pagination smoothly with proper links (self, first, next, last, prev), making it easy to navigate through customer resources.

Error handling is correct - when something goes wrong, the API returns the right status codes (400, 401, 403, 404, 429, 500, 504, 529) with clear messages so clients know what happened.

Performance is acceptable - responses come back in less than 2 seconds under normal conditions, providing a good user experience.

The API complies with LGPD regulations and Open Finance Brazil standards, protecting customer data and maintaining regulatory compliance.

By the end of this testing phase, we'll have confidence that the API Resources is production-ready and meets all functional, security, and performance requirements.

2. Version

Version: 1.0

Date: 06/22/2026

Author: Rodrigo Souza

Company: Klavi

Specification: Open Finance Brazil v3.0.0

Status: Ready for Execution

Last Updated: 06/24/2026

This test plan follows the ISO/IEC/IEEE 29119-3 standard for software testing documentation and covers the API Resources v3.0.0 as specified by Open Finance Brazil.

The plan incorporates industry-proven testing methodologies and best practices for complex API testing scenarios, including functional validation, security testing, performance baseline establishment, and regression coverage.

3. Scope

IN SCOPE (What Will Be Tested)

We'll focus on thoroughly testing the API Resources endpoint to ensure it works reliably for customers and integrating institutions.

The API Endpoint Itself

We'll test GET /resources to make sure it returns the correct list of resources that customers have consented to share. We'll validate all the required headers like Authorization and x-fapi-interaction-id, as well as query parameters for pagination.

Proper Response Handling

We'll check that the API returns the right HTTP status codes - 200 when everything works, 400 when the request has problems, 401 when authentication fails, 403 when someone doesn't have permission, 404 when something isn't found, and so on. Each error should give clear feedback about what went wrong.

Data Accuracy and Formatting

We'll verify that resourceId, type, and status fields are correct and properly formatted. Dates should follow ISO 8601 format, and the list of resources should accurately reflect what the customer has authorized.

Navigation Through Results

When there are many resources, the API should paginate results properly with links to navigate (first page, next page, previous page, etc). We'll make sure pagination works smoothly and customers can easily browse through all their resources.

Security and Access Control

We'll validate that OAuth 2.0 authentication works correctly and that only institutions with the `RESOURCES_READ` permission can access this data. We'll also test that the API is protected against common attacks like SQL injection and XSS.

Performance Under Real Conditions

We'll test how the API performs when multiple customers are accessing it simultaneously. Response times should stay under 2 seconds during normal usage, and the system should gracefully handle traffic spikes.

Checking for Breaking Changes

Since the API recently updated from v2.1.0 to v3.0.0, we'll verify that the changes were made correctly and that nothing was broken in the process.

Regulatory Compliance

We'll ensure the API complies with LGPD (Brazilian data protection law) and Open Finance Brazil standards, so customer data remains protected and regulations are followed.

OUT OF SCOPE (What We're NOT Testing)

We're specifically focusing on the Resources API, so we won't be testing other APIs like Accounts, Loans, or Credit Card endpoints - those have their own test plans.

We won't test the web or mobile interface - that's handled separately. Our focus is on the backend API itself.

We won't run extreme load tests with tens of thousands of users - we'll test realistic scenarios that reflect actual usage patterns.

We won't test disaster recovery scenarios or what happens if the entire system fails - that's a separate infrastructure concern.

We won't test in the live production environment with real customer data - all testing happens in controlled sandbox environments with test data.

4. Test Environments

We run tests across multiple environments, each serving a specific purpose in our quality assurance process. Here's how we use each one.

DEV Environment

This is where we start. Developers push code here and we run initial tests to catch problems early. We use synthetic test data and all external systems are mocked - meaning we're not actually calling real services, just simulating them. This keeps tests fast and isolated from other systems. Dev environment can be reset at any time, so developers can experiment freely without worrying about breaking anything.

Base URL: [DEV_BASE_URL]

Endpoint: GET /v3/resources

When Used: Daily during development, as soon as code is committed

Data: Test customers and resources, no real PII

External APIs: All mocked (OAuth, Accounts, Loans, etc)

Reset Policy: Can be reset anytime

STAGING Environment

Once features are working in dev, they move to staging.

This environment is much closer to production - it has real external integrations and data that mirrors what we see in production (but anonymized so no real customer information). Staging is where we do comprehensive testing before releasing to customers. We're careful with staging because it's the last stop before production.

Base URL: [STAGING_BASE_URL]

Endpoint: GET /v3/resources

When Used: Before each release, integration testing

Data: Production-like data (anonymized, no real customer info)

External APIs: Real connections or high-fidelity mocks

Stability: Should be stable, reset weekly

User Acceptance Testing (UAT) Environment

This is the UAT environment. Stakeholders and business teams use this to validate that the feature works as expected from a business perspective. It's very close to production and uses realistic scenarios. We typically keep this stable for 1-2 weeks so stakeholders can thoroughly test before we release to real customers.

Base URL: [UAT_BASE_URL]

Endpoint: GET /v3/resources

When Used: UAT phase, before production release

Data: Realistic test scenarios

External APIs: Sandbox versions of real APIs

Duration: Usually 1-2 weeks per release cycle

PRODUCTION Environment

This is the live environment where real customers are using the API. We're very conservative here - we only run smoke tests (quick checks that critical features work). We never do heavy testing or experimentation in production. All data is real customer data protected by LGPD regulations.

Base URL: [PRODUCTION_BASE_URL]

Endpoint: GET /v3/resources

When Used: Post-deployment validation only

Data: Real customer data (strictly protected)

External APIs: All real integrations

Testing: Smoke tests only (happy path scenarios)

Safety: Production is read-only for testing purposes

External API Dependencies

The Resources API doesn't work in isolation - it needs to call several other systems to do its job. Here's what those are and how we handle them in each environment.

In DEV, we mock everything. This means when the API tries to call the OAuth server, Accounts API, Loans API, etc, we have test doubles that return fake but realistic data. This makes tests fast and independent.

In STAGING, we use real integrations where possible. The OAuth server is real, the Consent system is real, and we have access to staging versions of other APIs.

In UAT, we use sandbox versions of all external systems - still not production, but realistic enough for stakeholder testing.

In PRODUCTION, everything is real - real OAuth tokens, real account data, real integrations with all external systems.

The main external dependencies are:

OAuth Authorization Server - validates customer tokens and checks permissions

Consent Management System - verifies that the customer actually authorized this data sharing

Accounts API - provides information about regular bank accounts

Credit Card Accounts API - provides credit card account information

Loans API - provides loan account information

Financing API - provides financing contract information

Investments APIs - provides investment account information

Exchange API - provides currency exchange operation information

For our testing, mocking these dependencies in DEV and STAGING means we can test the Resources API in complete isolation. We can control exactly what data comes back, test error scenarios easily, and make sure everything works before it reaches production.

5. Project Management & Agile Approach

Agile Methodology

We use Scrum with two-week sprints, Planning Poker for estimation, and daily standups every day until 11:00 AM Brazil Time. Every piece of code needs approval from 2 people before it goes live.

Sprint Duration: 15 days

Daily Standup: 15 minutes, until 11:00 AM Brazil Time

Code Review: Minimum 2 approvals required

Estimation: Planning Poker during sprint planning

Team Roles:

Product Owner

They represent the business, define what we test, and make the final call on releases. Available to answer questions and clarify requirements.

Scrum Master

Any dev team member can be Scrum Master. They facilitate meetings, remove blockers, and keep the team moving. It's about process, not technical decisions.

SDET / QA Tech Lead (Rodrigo Souza)

You define the testing strategy from day one. You decide what gets automated, what stays manual, what gets tested at all. You run performance tests, do code reviews, write unit tests alongside developers, and own the quality gate. You're also one of the 2 people who must approve all code before it ships.

QA Manual Tester

You execute tests, create test case documentation, find edge cases, and document bugs with clear steps to reproduce. You prepare test scenarios for automation so the SDET can make them automatic later. You need basic knowledge of automation and performance testing to understand what's happening under the hood.

Tech Lead Dev

Senior developer who reviews code for quality and architecture. They fix bugs we find and provide technical support. They're the second required approver for all test code.

How Code Review Works

Simple: Nothing merges without approval from both the SDET and Tech Lead Dev. The SDET checks if the test logic is solid and follows our strategy. The Tech Lead checks if the code is clean and performant. Both must say yes.

Code Review Approvals Required: 2 minimum

1. SDET - validates test approach
2. Tech Lead Dev - validates code quality

This ensures quality from both sides.

6. Risks

R-001: External API Dependencies Down

Risk: OAuth server, Consent API, Accounts API, or other external dependencies become unavailable during testing.

Probability: Medium

Impact: High

Mitigation Actions (Concrete):

- SDET configures mocks in DEV for 100% of APIs (Wiremock)
- STAGING uses real APIs where possible, mock fallback if unavailable
- Establish contact with team responsible for each API (Slack channel)
- Schedule monthly sync to communicate maintenance windows
- Setup Grafana alerts to monitor health of each external API
- If API goes down: switch to DEV+mocks within maximum 30 minutes
- Document in Confluence: "API X unavailable, use these tests in DEV"
- Ownership: SDET responsible for validating mocks every sprint

R-002: Database or Microservice Failures

Risk: Database slow/crash or microservices (Accounts, Loans) become unavailable.

Probability: Medium

Impact: High

Mitigation Actions:

- Coordinate with infrastructure: which microservices have SLA?
- Document: which test depends on which microservice
- Implement timeout in tests: fail fast if service is slow
- Have pre-loaded test data (doesn't depend on complex queries)
- Setup monitoring: alert if database latency > 500ms
- Plan B: execute smoke tests offline when dependencies fail
- Weekly review: check failure logs, adjust tests
- Ownership: Tech Lead Dev + SDET monitor

R-003: System Unavailability During Test Windows

Risk: Test environment goes down during test execution (maintenance, crash).

Probability: Medium

Impact: High

Mitigation Actions:

- Coordinate with infrastructure: what are maintenance windows?
- Schedule tests outside these windows (ex: 2pm-5pm, never overnight)
- Use DEV as immediate fallback if STAGING goes down
- Setup nagios/Grafana alert: notify Slack within 2 minutes if down
- SLA with infrastructure: restore environment within 30 minutes max
- If environment down 1+ hour: escalate to PO (may delay release)
- Document procedure: "Environment crashed, follow steps X, Y, Z"
- Ownership: SDET monitors during critical test runs

R-004: Development Team Resource Constraints

Risk: Dev/QA becomes unavailable (illness, departure) and schedule becomes tight.

Probability: Medium

Impact: High

Mitigation Actions:

- EVERYTHING documented in Confluence (not emails or heads)
- Mandatory code review: knowledge always shared
- Pair programming 2 hours per week on critical topics
- Clear prioritization: critical tests first (happy path)
- If someone leaves: another can pick up task within 1 day
- Schedule with buffer: add 3 extra days before release
- Communication: if resources get tight, PO must know IMMEDIATELY
- Ownership: SDET manages knowledge via continuous documentation

R-005: Inadequate Test Data

Risk: Test data doesn't cover real scenarios or lacks volume for performance testing.

Probability: Medium

Impact: Medium

Mitigation Actions:

- Sprint 1: work with PO to define complete dataset
- Collect anonymized data from production (if possible)
- Create 3 scenarios: happy path + edge cases + volume test
- Performance testing: test with 100, 1,000, 5,000+ records
- Document in Confluence: which data tests which scenario
- Validate data with Tech Lead Dev before using
- Monthly review: new scenarios found, update data
- Ownership: QA Manual Tester responsible for data

R-006: Authentication & Credential Management

Risk: Credentials expire, OAuth tokens not generated correctly, access rejected.

Probability: High

Impact: High

Mitigation Actions:

- Store credentials in Vault (not hardcoded in code)
- SDET implements auto-refresh: token refreshed 30min before expiry
- Have 2+ test users as backup
- Alert: 7 days before credential expires (Jira ticket)
- SLA: credential renewed within maximum 24 hours
- Test token refresh logic in CI pipeline itself
- Setup: test credential auto-renewed at every sprint
- Documentation: where to obtain credentials, validity duration
- Ownership: Tech Lead Dev + SDET manage

R-007: Integration Testing Complexity

Risk: Difficult to test integration with multiple APIs without controlled environment.

Probability: Medium

Impact: Medium

Mitigation Actions:

- DEV: 100% mocked, fast and isolated tests
 - STAGING: real APIs + mocks depending on availability
 - UAT: real tests with sandbox APIs (pre-coordinate)
 - Document: expected behavior of each integration
 - Clear prerequisites: which API needs to be up for which test
 - Run integration tests only if all APIs are healthy
 - Monitor status of each integration in real time
 - Ownership: SDET validates integrations, QA Manual tests end-to-end
-

7. Test Strategy

Our testing approach balances automation and manual testing to catch bugs early while keeping execution fast.

Testing Pyramid

Most tests are unit tests (done by devs). Middle layer is integration and API tests (automated). Top is manual/exploratory (catch edge cases humans find).

Unit Tests (Dev Responsibility)

- SDET iterates with devs from day 1
- Every function gets tested before it's coded
- Runs in pipeline: < 1 minute
- Goal: 85%+ code coverage

API Contract Tests (SDET - Automated)

- Validate schema matches specification
- Test all HTTP status codes
- Verify response structure and data types
- Runs in pipeline: ~3 minutes
- Tools: Rest Assured

Functional Tests (SDET - Automated)

- Happy path: does the API do what it's supposed to?
- Edge cases: malformed requests, boundary values
- Error handling: proper status codes and messages
- Runs in pipeline: ~8 minutes per environment
- Tools: Rest Assured, parameterized tests

Security Tests (SDET + Manual)

- OAuth validation, scope checking
- SQL injection, XSS prevention
- Token manipulation detection
- Runs in STAGING before release
- Tools: Burp Suite (manual), Rest Assured (automated checks)

Performance Tests (SDET - Automated)

- Load test: 50 concurrent users for 2 minutes
- Spike test: sudden jump from 20 to 100 users
- Response time: must be < 2 seconds p95
- Error rate: must be < 1 %
- Runs weekly in STAGING
- Tools: k6

Manual/Exploratory Tests (QA Manual)

- Test like a real customer would
- Find unexpected behaviors
- Test across different scenarios and sequences
- Validate error messages make sense
- Check pagination and data accuracy
- Runs in STAGING before release
- Documentation: test cases in Confluence

Regression Tests (QA Manual + SDET)

- Before each release: verify nothing broke
- Run previous sprint's tests again
- Automated where possible, manual for edge cases
- Goal: ensure quality from sprint 1 to release

What Gets Automated vs What Stays Manual

AUTOMATED:

- API contract validation (schema, status codes)
- Happy path scenarios (standard customer usage)
- All 41 test cases from test matrix
- Performance baselines
- Regression tests (reuse from previous sprints)
- Anything that runs more than 2 times

MANUAL:

- Exploratory testing (find unexpected things)
- Complex business scenarios (need human judgment)
- Edge cases discovered during testing
- Usability of error messages
- Test data preparation and setup
- Bug validation and reproduction

HYBRID:

- Security testing: SDET does automated checks, manual does deep dive
- Performance: k6 for baseline, manual investigation if slow
- Regression: automated regression suite + manual sanity check

Test Execution Frequency

DEV Environment (Every Commit)

- Unit tests: automatic via pipeline
- Contract tests: automatic via pipeline
- Security checks: automatic via pipeline
- Time: < 5 minutes total

STAGING Environment (Daily + Before Release)

- All automated tests run
- Manual smoke testing by QA
- Performance baseline check
- Regression test suite
- Time: 20-30 minutes

UAT Environment (Release Window)

- Manual validation by stakeholders
- Smoke tests of critical features
- Final sign-off before production
- Time: varies based on stakeholder availability

PRODUCTION (Post-Deployment)

Smoke tests only: just verify critical features work.

Time: 5 minutes.

Prioritization: What to Test First

Sprint 1-2: Auth works, API returns 200, schema matches, all status codes correct.

Sprint 3-4: Pagination works, all 12 resource types work, security checks pass, performance baseline set.

Sprint 5+: Edge cases, error messages, complex scenarios, full regression.

Test Data Strategy

DEV: 5 test customers, 50 synthetic resources

STAGING: 20 test customers, 200+ resources, anonymized

UAT: Real-world scenarios, stakeholder understands

Success Criteria

Pass if: 100% critical tests pass, 95% high-priority pass,

0 flaky tests, 0 blocker bugs, < 2 seconds response time, 85%+ code coverage.

8. CI/CD Pipeline

Every code change automatically runs tests. If tests fail, code doesn't go to production.

How It Works

Developer commits → Tests run automatically → If pass, can merge → If fail, fix it and try again.

Takes about 30 minutes total from commit to production (if everything passes).

What Tests Run Automatically

DEV: Unit tests, contract tests, basic security checks
(< 8 minutes)

STAGING: All 41 functional tests, performance check, regression tests (< 20 minutes)

PRODUCTION: Smoke tests, health checks, performance baseline
(< 5 minutes)

QA Test Types in Pipeline

Smoke Tests: Quick verification that critical features work
(GET /resources returns 200, pagination works, headers correct)

Health Checks: Verify API is alive and responding
(endpoint available, no 5xx errors, response time < 3 seconds)

Performance Tests: Load testing using K6 or Artillery
(50 concurrent users, response time < 2 seconds p95, error rate $< 1\%$)

When Tests Fail

If DEV tests fail: Developer can't merge, must fix.

If STAGING tests fail: Code doesn't go to production until fixed.

If PROD smoke test or health check fails: We rollback to previous version immediately.

Deployment Timeline

Code written → Code review (2 people approve) → Tests run
→ Production → Verify it works (smoke test + health check + performance baseline)

Total time: ~2 hours if everything passes.

9. Tools and Infrastructure

We use tools to write tests, track progress, monitor quality, and deploy safely. Specific choices to be defined with Klavi.

Testing & Automation Framework

API Test Automation Tool (To be determined with Klavi)

- Options: Rest Assured, Playwright, Cypress, or similar
- Must support automated API testing
- Must integrate with CI/CD pipeline
- Must generate clear test reports

Programming Language (To be determined with Klavi)

- Options: Java, Python, JavaScript
- Team preference and Klavi standards will guide choice
- Must be maintainable and scalable

Performance Testing

Performance Testing Tool (To be determined with Klavi)

- Options: K6, Artillery, JMeter
- Load testing: simulate concurrent users
- Must generate performance reports and baselines

Code Quality Monitoring

SonarQube or Similar

- Measure test code coverage (goal: 85%+)
- Identify code issues
- Track technical debt

Grafana or Similar

- Visualize test trends over time
- Performance baselines and monitoring
- Real-time dashboards

Project & Documentation

Jira

- Track test cases, bugs, tasks
- Sprint planning and execution
- Two-person approval gate for code

Confluence

- Test plan documentation
- Test procedures and guides
- Team decisions and knowledge base

Infrastructure & Deployment

Cloud Platform (AWS or Azure)

- Host DEV, STAGING, UAT environments
- Database and microservice management

Source Control & CI/CD (GitLab or GitHub)

- Repository for code and tests
- Automated pipeline execution
- Deployment automation

API Documentation & Testing

Swagger or OpenAPI

- API specification and documentation
- Schema validation

Postman, Insomnia, or Similar

- Manual API testing
 - Request/response debugging
 - Test data preparation
-

10. Activities & Estimates

Timeline: 12 weeks (4 sprints of 15 days each)

SPRINT 1 (Days 1-15): Foundation

Design test plan and framework

Setup CI/CD pipeline

Create first batch of tests

Document procedures in Confluence

Effort: 40 hours (SDET) + 20 hours (QA Manual)

SPRINT 2 (Days 16-30): Implementation

Write all 41 automated test cases

Manual testing and exploratory testing

Setup performance baseline (K6)

Fix and integrate tests

Effort: 40 hours (SDET) + 40 hours (QA Manual)

SPRINT 3 (Days 31-45): Hardening

Full regression testing

Security testing

Prepare UAT environment

Final documentation and training

Effort: 40 hours (SDET) + 40 hours (QA Manual)

SPRINT 4+ (Post-Release): Maintenance

Monitor production

Fix issues found

Add new tests as needed

Ongoing support

Effort: ~10 hours/week per person

Key Deliverables

41 automated test cases

Performance baselines (K6)

Full regression suite

Complete documentation

CI/CD pipeline operational

Team trained on maintenance

11. Test Matrix

We have 41 test cases organized in Jira covering:

- Happy path scenarios (4 tests)
- Authentication and authorization (6 tests)
- Data validation (8 tests)
- Pagination and pagination (5 tests)
- HATEOAS links (3 tests)
- Content negotiation (3 tests)
- Rate limiting (2 tests)
- Error handling (3 tests)
- Performance (2 tests)
- Security (2 tests)
- Regression (2 tests)

All test cases are documented in Jira with expected results, acceptance criteria, and execution steps. Test case details available in Jira project: **[PROJECT_KEY]**

12. Communication Plan & Reports

Where we communicate and how we track issues.

Slack

Team channel: #klavi-qa-testing

- Test results and updates
- Blockers and urgent issues
- Daily progress posts
- All messages in English

Jira

Bugs found:

- Create ticket immediately
- Include: steps to reproduce, expected vs actual, screenshots
- Assign priority: Critical, High, Medium, Low
- Track status: Open → In Progress → Done

Test cases:

- Documented with expected results and acceptance criteria
- Execution status tracked
- All in English

Documentation

Confluence:

- Test plan and procedures
- How to run tests locally
- Troubleshooting guides
- All in English

Language Standard

All communication in English (Slack, Jira, Confluence, docs)

- Team is distributed
- Ensures clarity for everyone
- Project standard regardless of location